

Projet: UE MOGPL

Programmation linéaire

Groupe:

GROUPE 4

Auteur:

Lecene Maël

Version du document:

1.0

Sommaire

1	Introduction	3	10	Modèle Final : Le Mélange de Régimes . . .	14
2	Formulation du problème en graphe orienté	3	10.1	Analyse de la Transition de Phase ...	15
2.1	1. Définition des sommets (Espace des états)	3	10.2	5. Interprétation des 3 Régimes	15
2.2	2. Définition des arêtes (Transitions) .	3	11	Modélisation de la Distance Effective : La Divergence Critique	15
2.3	3. Fonction de coût	3	11.1	1. Le Facteur de Tortuosité $\tau(\rho)$	15
2.4	4. Objectif	4	11.2	2. Loi de Divergence Asymptotique .	16
3	Structure de Données : Représentation du Graphe	4	11.3	3. Conséquence sur le Modèle de Coût	16
3.1	1. Structure : Dictionnaire et Ensembles	4	12	Justification Physique du Modèle : Analogie avec la Percolation	16
3.2	2. Stratégie du « Graphe Implicite » (Abstraction des États)	4	12.1	1. Le Phénomène de Transition de Phase	17
4	Algorithmes de résolution	5	12.2	Modélisation du Coût Structurel de A^*	17
4.1	Analyse de la complexité	5	12.3	Analyse du Ratio de Performance R (Modèle Avancé)	17
4.2	1. Impact des États et Connectivité ...	5	13	Analyse des Performances (Benchmark) ..	18
4.3	2. Dimensionnement et Impact des Obstacles	5	13.1	1. Impact de la taille de la grille ($N \times M$)	18
4.4	3. Complexité Algorithmique	6	13.2	2. Impact de la densité locale (Grille 20×20)	19
5	Critère de Choix Optimal : Modélisation du Ratio de Performance (Bonus)	7	13.3	3. Analyse Avancée : Seuil de Percolation (Grille 100×1000)	20
6	Analyse du BFS	8	14	Modélisation du programme linéaire	21
6.1	1. Hypothèse du « Monde Infini »	8	14.1	Variables et paramètres	21
6.2	2. Analyse du Squelette (Terme Linéaire)	8	14.2	Modélisation du Problème de PLNE .	21
6.3	3. Analyse des Quadrants (Terme Quadratique)	8	14.3	Preuve de la contrainte Anti-motif 1 — 0 — 1	22
6.4	4. Synthèse et Borne	9	15	Bibliographie	23
7	1. Modélisation de la Densité : Obstacles et Percolation	9	15.1	Ouvrages de Référence	23
7.1	Impact Topologique Local	9	15.2	Ressources Multimédia	23
7.2	Limite de Validité : Seuil de Percolation	9	15.3	Supports Académiques Personnels . .	23
8	2. Analyse Géométrique Fine en Milieu Borné	10	16	Remerciements	23
8.1	Paramètres de l'Instance	10			
8.2	Synthèse : Espérance du Coût Global	11			
8.3	Conclusion et Modèle de Coût du BFS	11			
9	Analyse Fine A^* : (Cycle RC)	12			
9.1	1. Le Phénomène Physique	12			
9.2	2. Modélisation Mathématique (Fonction par Morceaux)	13			
9.3	3. Synthèse de la Fonction de Navigabilité	14			

1 Introduction

Dans le cadre de l'optimisation logistique d'un entrepôt automatisé, ce projet vise à minimiser le temps de transport d'un robot mobile circulant sur un réseau de rails en grille. Le robot, contraint par des déplacements rectilignes et la présence d'obstacles, doit rallier un point de départ à une destination donnée.

La complexité du problème réside dans la nature des commandes disponibles : le robot peut avancer de 1, 2 ou 3 mètres ou effectuer des rotations de 90° vers la gauche ou vers la droite, chaque action consommant une seconde. L'objectif est d'optimiser la séquence d'actions pour réduire le coût temporel total.

Ce travail aborde trois axes principaux :

- ▶ La modélisation du problème sous forme de graphe orienté.
- ▶ L'implémentation et l'analyse de complexité d'un algorithme de résolution.
- ▶ La génération aléatoire d'obstacles via la programmation linéaire (Gurobi).

2 Formulation du problème en graphe orienté

Nous modélisons l'entrepôt et les mouvements du robot par un graphe orienté $G = (V, E, w)$.

2.1 1. Définition des sommets (Espace des états)

Soit $\mathcal{O} = \{N, S, E, O\}$ l'ensemble des quatre orientations cardinales possibles. Nous définissons l'ensemble des sommets V comme l'ensemble des triplets (l, c, o) représentant une configuration valide du robot :

$$V \subseteq \{0, \dots, M-1\} \times \{0, \dots, N-1\} \times \mathcal{O}$$

Un sommet $v = (l, c, o) \in V$ existe si et seulement si la position (l, c) est physiquement accessible (c'est-à-dire que le robot ne collisionne avec aucun obstacle en étant centré sur cette intersection).

2.2 2. Définition des arêtes (Transitions)

L'ensemble des arêtes E représente les commandes possibles. Soit $u = (l, c, o) \in V$ un état courant. Les arcs partants de u sont définis par les actions suivantes :

Arêtes de Rotation (Tourne) :

Ces transitions sont dynamiques et dépendent de l'état courant. Elles ne correspondent pas à des rails physiques fixes, mais à des actions virtuelles permettant de changer l'orientation sur place. Depuis un état (l, c, o) , des arêtes sont générés vers :

$$(l, c, o_{\text{gauche}}) \quad \text{et} \quad (l, c, o_{\text{droite}})$$

où o_{gauche} et o_{droite} sont les orientations adjacentes à o (changement de 90°).

Arêtes de Mouvement (Avance) :

Pour tout $k \in \{1, 2, 3\}$, le robot peut avancer de k mètres. Si la position (l', c') est atteinte après avoir avancé de k pas dans la direction o sans rencontrer d'obstacle, alors il existe un arête:

$$(l, c, o) \longrightarrow (l', c', o)$$

2.3 3. Fonction de coût

Puisque chaque commande (Avance ou Tourne) prend strictement 1 seconde, le graphe est uniformément pondéré. La fonction de poids $w : A \rightarrow \mathbb{R}^+$ est définie par :

$$\forall e \in A, \quad w(e) = 1$$

2.4 4. Objectif

Soit $s_{\text{start}} = (l_{\text{start}}, c_{\text{start}}, o_{\text{start}})$ l'état initial donné. Soit T un état cible, défini par la position d'arrivée $(l_{\text{end}}, c_{\text{end}})$ pour toute orientation o :

$$T = \{(l, c, o) \in V \mid l = l_{\text{end}}, c = c_{\text{end}}\}$$

Le problème revient à trouver le chemin de coût minimum dans le graphe G partant de s_{start} et atteignant l'un des états de T . Il est à noter que, pour le graphe d'état du système, il s'agit d'un graphe orienté. Cependant, par la suite, notre modélisation se base sur un graphe non orienté pour des raisons d'optimisation.

3 Structure de Données : Représentation du Graphe

Le choix de la structure de données pour représenter l'environnement de navigation est déterminant pour les performances de l'algorithme (A^* ou BFS). Nous avons écarté la représentation matricielle classique (Grille 2D) et la représentation explicite de l'espace d'états au profit d'une approche hybride : une **Liste d'Adjacence Hashée**.

3.1 1. Structure : Dictionnaire et Ensembles

Le graphe G est stocké sous la forme d'un dictionnaire.

- **Clé** : Un tuple représentant les coordonnées spatiales (u, v) .
- **Valeur** : Un ensemble (Set) contenant les coordonnées des voisins accessibles.

$$G: \text{Map}<(x, y), \text{Set}<(x_i, y_i)>>$$

Ce choix technique répond à trois impératifs :

1. **Complexité d'accès $O(1)$** : L'utilisation d'un Set pour les voisins permet de vérifier l'existence d'une connexion ou d'un obstacle en temps constant moyen, contrairement à une liste linéaire $O(k)$.
2. **Compacité** : Contrairement à une matrice d'adjacence $O(V^2)$ qui gaspillerait de la mémoire pour stocker des « vides », le dictionnaire ne stocke que les nœuds existants et leurs connexions réelles. Pour un graphe creux (comme un labyrinthe), le gain spatial est massif ($O(V + E)$).
3. **Mutabilité des Clés** : Le tuple (u, v) est une structure légère et immuable en tant que clé, mais le dictionnaire permet d'ajouter dynamiquement des nœuds si la topologie change, sans réallouer un tableau complet.

3.2 2. Stratégie du « Graphe Implicite » (Abstraction des États)

Une contrainte majeure de notre système est la gestion des états complexes. **Problème** : Si nous devons stocker chaque état possible dans la structure de données (ex: nœud (x, y, θ)), l'explosion combinatoire de la mémoire rendrait le système très lourd.

Solution : Le Graphe Statique vs Sommets Virtuels Nous avons choisi de ne stocker que le **Graphe Simple** (la géométrie pure du terrain) dans G . Les états ne sont pas pré-calculés mais générés à la volée par l'algorithme de recherche.

- **Le Graphe G (Physique)** : Il ne contient que la vérité terrain (« La case A est connectée à la case B »). Il est agnostique de l'agent.
- **L'Algorithme (Logique)** : C'est l'algorithme qui instancie des **sommets virtuels intermédiaires**. Lorsqu'il explore un nœud, il interroge G pour connaître les voisins physiques, puis applique ses règles de transition pour filtrer ou pondérer ces voisins.

Avantage Décisif : Cette séparation des responsabilités permet au modèle de se concentrer sur l'essentiel. Le graphe reste léger et rapide à interroger.

4 Algorithmes de résolution

Pour résoudre ce problème, nous avons implémenté deux algorithmes complémentaires afin de tirer parti de la topologie du graphe :

- ▶ **BFS (Breadth-First Search) :** Le premier algorithme est un parcours en largeur.
- ▶ **A-Star :** Le second est un algorithme A-Star avec l'heuristique de Manhattan.

4.1 Analyse de la complexité

Cette section détaille la modélisation mathématique du problème et justifie le choix des algorithmes par une analyse asymptotique.

4.2 1. Impact des États et Connectivité

La définition du robot impose une topologie spécifique où chaque intersection physique se décline en 4 états (x, y, θ) .

Contrairement à une grille classique, le robot dispose de **commandes macroscopiques** : il peut avancer de 1, 2 ou 3 cases en une seule action (coût unitaire). Le degré sortant théorique d'un sommet interne est donc de $k = 5$:

- ▶ **3 Transitions Spatiales :** Avance(1), Avance(2), Avance(3).
- ▶ **2 Transitions Angulaires :** Tourne(Gauche), Tourne(Droite).

Note (Effet de Paroi) : Ce degré de 5 constitue une **borne supérieure** valable pour les sommets centraux. Les sommets situés en bordure de grille ou proches d'obstacles possèdent un degré inférieur (mouvements longs impossibles).

4.3 2. Dimensionnement et Impact des Obstacles

L'analyse est effectuée sur le **sous-graphe induit des intersections**. Nous démontrons ici que l'ajout d'obstacles diminue la densité globale.

4.3.a a) Suppression des Sommets ($|V|$)

Soit V_0 le nombre d'intersections physiques du graphe initial (grille de taille $N \times M$) :

$$V_0 = (N - 1) \times (M - 1)$$

Le graphe d'états totalise $4V_0$ sommets (4 orientations par intersection).

L'ajout d'un obstacle physique supprime un bloc de 4 intersections. La réduction du nombre de sommets est donc :

$$|V|(P) = 4V_0 - 16P$$

Contrainte de validité : Le nombre d'obstacles P est physiquement borné par la surface disponible. On ne peut avoir un nombre de sommets négatif.

$$16P \leq 4V_0 \rightarrow P \leq \frac{V_0}{4}$$

4.3.b b) Suppression des Connexions ($|E|$)

Il est crucial de distinguer la **Somme des Degrés** (les extrémités de connexions) du nombre d'Arêtes $|E|$ (les liens physiques). La somme des degrés vaut deux fois le nombre d'arêtes (Sum deg = $2 |E|$).

1. **Somme des Degrés Initiale** : Avec un degré moyen initial de 5, la somme totale est $5 \times 4V_0 = 20V_0$.
2. **Perte de Connectivité** : Lorsqu'on retire un bloc, on supprime :
 - ▶ Les connexions internes et sortantes des 16 sommets supprimés ($16 \times 5 = 80$).
 - ▶ Les connexions entrantes venant des voisins survivants (estimées à 24 arcs coupés aux interfaces).

$$\Delta(\text{Sum deg}) \approx 104 \text{ extrémités d'arcs}$$

3. **Nombre d'Arêtes Résiduel** : On divise la somme des degrés par 2 pour obtenir le nombre d'arêtes physiques.
 - ▶ Initial : $10V_0$ arêtes.
 - ▶ Perte par obstacle : $\frac{104}{2} = 52$ arêtes.

$$|E|(P) = 10V_0 - 52P$$

4.3.c c) Preuve de la décroissance du degré moyen (δ)

Le coût unitaire de l'algorithme dépend du degré moyen résiduel $\delta(P)$.

$$\delta(P) = \frac{\text{Sum deg}(P)}{|V|(P)} = \frac{2 \times |E|(P)}{|V|(P)}$$

En remplaçant par les expressions en fonction de P :

$$\delta(P) = \frac{2(10V_0 - 52P)}{4V_0 - 16P} = \frac{20V_0 - 104P}{4V_0 - 16P}$$

Analyse du Ratio Marginal : Pour comprendre la dynamique, observons le rapport de suppression :

$$\text{Ratio} = \frac{\Delta \text{ Numérateur}}{\Delta \text{ Dénominateur}} = \frac{-104}{-16} = 6.5$$

Conclusion : Puisque chaque obstacle supprime « plus de connexions que de sommets » (Ratio de 6.5), le graphe s'appauvrit structurellement. Le degré moyen est donc une fonction strictement **décroissante** de P sur l'intervalle de validité $\left[0, \frac{V_0}{4}\right]$.

4.4 3. Complexité Algorithmique

Les complexités s'expriment en fonction de la taille de ce graphe d'états étendu.

4.4.a BFS

La complexité du BFS est linéaire par rapport à la taille du graphe. Elle se décompose en deux phases : l'extraction de chaque sommet et le parcours de ses voisins.

Le coût total est donc la somme du coût de traitement de chaque sommet visité :

$$T_{\text{BFS}} = \sum_{v \in V} (1 + \deg(v)) = \underbrace{|V|}_{\text{Sommets}} + \underbrace{\sum_{v \in V} \deg(v)}_{\text{Transitions (|E|)}} = O(|V| + |E|)$$

Le terme $|E|$ provient directement de l'accumulation des degrés, le BFS explore systématiquement toutes les rotations possibles, même si elles tournent le dos à l'objectif.

4.4.b Algorithme A-Star (A^*)

La complexité temporelle de A^* dans le pire des cas est donnée par :

$$T_{A^*} = O(|E| \log |V|)$$

Rq : Ici, on peut démontrer assez trivialement que m est un majorant de n , donc le n disparaît dans la notation de Landau, ce qui explique le résultat.

Cette borne théorique s'explique par la structure de l'algorithme :

- ▶ **Exploration exhaustive ($|E|$) :** Dans le pire des cas (si l'heuristique est non-informative ou si le chemin est très complexe), l'algorithme doit explorer et relâcher la totalité des arêtes du graphe pour garantir l'optimalité.
- ▶ **Coût de structure ($\log |V|$) :** Chaque fois qu'une arête est explorée, il peut entraîner une mise à jour de la priorité d'un sommet dans la file. Dans un tas binaire, le coût de maintenance de l'ordre (insertion) est logarithmique.

Le Rôle de l'Heuristique : En pratique, l'algorithme A^* utilise une fonction de coût $f(n) = g(n) + h(n)$ qui agit comme un filtre :

- ▶ **Arêtes Prometteurs :** Si une transition rapproche le robot de la cible, $h(n)$ diminue. Le coût total $f(n)$ reste bas, ce qui place l'état en tête de file pour être traité immédiatement.
- ▶ **Arêtes Pénalisantes :** À l'inverse, si une transition éloigne le robot ou implique un coût élevé (rotation), $g(n)$ augmente sans baisse compensatoire de $h(n)$. Le score $f(n)$ augmente, reléguant cet état au fond de la file.

Conséquence : Bien que la complexité pire cas dépende de la totalité des arêtes $|E|$, l'heuristique permet en réalité de n'en visiter qu'une infime fraction. Les transitions « contre-intuitives » restent en attente au fond de la file de priorité et ne sont jamais traitées tant qu'un chemin direct semble viable. Le nombre d'arêtes effectivement explorées s'effondre, rendant l'algorithme drastiquement plus rapide que sa borne théorique.

Synthèse et limites : Bien que le coût unitaire des opérations de A^* soit plus élevé à cause du facteur $\log |V|$, cet algorithme reste souvent plus performant sur les graphes denses en pratique. L'heuristique permet en effet d'élaguer drastiquement l'espace de recherche, évitant ainsi de parcourir la totalité des arêtes $|E|$.

Afin de déterminer précisément le point de bascule entre l'efficacité brute du BFS et l'intelligence heuristique de A^* , nous nous proposons de modéliser plus finement les temps d'exécution dans la section suivante.

5 Critère de Choix Optimal : Modélisation du Ratio de Performance (Bonus)

Notre objectif est de déterminer formellement le seuil de bascule entre l'algorithme BFS et l'algorithme A^* . Plutôt que de nous reposer uniquement sur la notation grand-O, nous cherchons à modéliser le coût espéré précis pour définir un ratio critique.

Nous posons l'inégalité de décision suivante :

$$\frac{E[\text{Cost}]_{\text{BFS}}}{E[\text{Cost}]_{A^*}} > 1 \text{ (choisi le } A^*)$$

Où le coût espéré d'un algorithme est le produit du nombre moyen de sommets traités par le coût unitaire de traitement d'un sommet :

$$E[\text{Cost}] = E[V_{\text{traité}}] \times C_{\text{unitaire}}$$

Dans les sections suivantes, nous construisons une expression analytique fine de $E[V_{\text{traité}}]$ pour le BFS, en intégrant successivement l'impact des obstacles (densité) et la géométrie de l'entrepôt (murs).

6 Analyse du BFS

Pour cette analyse, nous nous concentrerons sur le graphe physique, sans tenir compte des états, qui ne représentent qu'un facteur multiplicatif.

6.1 1. Hypothèse du « Monde Infini »

Pour établir la complexité intrinsèque de l'algorithme, nous nous plaçons dans le cadre d'une **grille infinie** (ou d'un entrepôt suffisamment grand tel que $M, N \gg d$).

Cette hypothèse est fondamentale pour deux raisons :

- Elle permet d'ignorer les **effets de bord** : les quadrants de la zone de recherche ne sont jamais tronqués par les murs de l'entrepôt.
- Elle garantit que la frontière de recherche $N(d)$ peut s'étendre librement sans saturation.

Dans ce contexte non borné, l'ensemble des points accessibles en moins de d étapes forme un losange géométrique parfait. Nous décomposons cet ensemble $S(d)$ en trois parties :

1. Le **Centre** (point de départ, constante 1).
2. Le **Squelette** (les 2 axes orthogonaux).
3. Les **quadrants** (les 4 intérieurs).

$$S(d) = 1 + S_{\text{axes}}(d) + S_{\text{quadrants}}(d)$$

6.2 2. Analyse du Squelette (Terme Linéaire)

Les axes représentent les déplacements purement cardinaux sans changement de direction. Puisque le robot peut avancer de 3 cases maximum en un coup ($k = 3$), chaque branche cardinale s'allonge de 3 nouvelles unités à chaque incrément de profondeur d . Avec 4 directions cardinales, le nombre de sommets sur les axes est :

$$S_{\text{axes}}(d) = 4 \times (3 \times d) = 12d$$

Ce terme est linéaire car les axes n'ont pas de « largeur », ils ne font qu'avancer.

6.3 3. Analyse des Quadrants (Terme Quadratique)

C'est ici que réside la complexité majeure. Un quadrant est l'espace compris entre deux axes (ex: entre Nord et Est). Les nouveaux nœuds apparaissent pour combler l'écart grandissant entre ces axes.

Soit $N_{\text{quad}}(i)$ le nombre de nouveaux sommets ajoutés dans **un seul** quadrant à l'étape i . L'écart entre les axes augmente de 3 cases à chaque étape. La « frontière » intérieure suit donc une suite arithmétique.

- À l'étape $i = 1$, l'intérieur est vide ($N_{\text{quad}}(1) = 0$).
- À chaque étape suivante, la longueur du segment reliant les axes augmente de 3 cases.

La suite régissant le remplissage d'un quadrant est :

$$N_{\text{quad}}(i) = 3 \times (i - 1)$$

Pour obtenir le total des nœuds dans les 4 quadrants à la profondeur d , nous sommions cette suite et multiplions par 4 :

$$S_{\text{quadrants}}(d) = 4 \times \sum_{i=1}^d 3(i - 1)$$

Par linéarité de la somme, nous factorisons par 12 :

$$S_{\text{quadrants}}(d) = 12 \times \sum_{j=0}^{d-1} j$$

En utilisant la formule de la somme des entiers $\sum k = \frac{n(n+1)}{2}$:

$$S_{\text{quadrants}}(d) = 12 \times \frac{(d-1)d}{2} = 6d(d-1) = 6d^2 - 6d$$

6.4 4. Synthèse et Borne

En recombinant les termes calculés précédemment (Centre + Axes + Quadrants) :

$$S(d) = 1 + (12d) + (6d^2 - 6d)$$

$$S(d) = 6d^2 + 6d + 1$$

Nous concluons par le résultat fondamental suivant :

$$S(d) = 6d^2 + 6d + 1$$

Cette formule démontre que le nombre d'états visités croît de manière **quadratique** :

$$S(d) \in \theta(d^2)$$

Cela justifie l'utilisation du BFS pour des distances raisonnables, car l'algorithme ne souffre pas d'une explosion combinatoire exponentielle.

7 1. Modélisation de la Densité : Obstacles et Percolation

Avant d'analyser la surface géométrique, il faut qualifier la viabilité des sommets. L'entrepôt n'est pas vide : il contient des obstacles répartis aléatoirement selon une densité ρ on pose :

$$\rho = \frac{4 \times P}{N \times M}$$

7.1 Impact Topologique Local

Comme établi précédemment, l'ajout d'un obstacle physique dans une case de la grille n'est pas ponctuel. Il entraîne la suppression du **bloc de 4 sommets** adjacents dans le graphe de résolution.

Soit X_i une variable de Bernoulli représentant la présence d'un obstacle sur le sommet i . Si la densité d'obstacles physiques est ρ , la probabilité qu'un sommet donné soit invalide (supprimé) est approximée par 4ρ (car dépendant de 4 cases physiques).

7.2 Limite de Validité : Seuil de Percolation

Cette approximation linéaire n'est valide que si le graphe reste globalement connexe. Il existe une densité critique ρ_c (seuil de percolation) :

- Si $\rho < \rho_c$: Le graphe contient une « composante géante ». L'impact des obstacles est une simple réduction linéaire du volume de recherche.
- Si $\rho > \rho_c$: L'espace se fragmente en clusters isolés. L'heuristique de A s'effondre (pièges locaux), et le BFS sature rapidement les petits clusters.

Hypothèse de travail : Pour la suite, nous nous plaçons dans le régime sous-critique ($\rho < \rho_c \approx 0.1$) où le chemin existe et où la réduction d'espace est modélisable par Bernoulli.

8 2. Analyse Géométrique Fine en Milieu Borné

Nous cherchons maintenant à dénombrer les sommets géométriquement accessibles $S(d)$ en tenant compte des murs de l'entrepôt. Nous abandonnons l'approximation globale par quadrant au profit d'une **décomposition axiale**.

8.1 Paramètres de l'Instance

Soit (l_0, c_0) la position de départ et $(l_{\text{end}}, c_{\text{end}})$ la cible. Les capacités de déplacement maximales (en nombre d'actions de 3 mètres) avant de heurter un mur sont :

$$\delta_N = \left\lceil \frac{l_0}{3} \right\rceil, \quad \delta_S = \left\lceil \frac{M - 2 - l_0}{3} \right\rceil$$

$$\delta_O = \left\lceil \frac{c_0}{3} \right\rceil, \quad \delta_E = \left\lceil \frac{N - 2 - c_0}{3} \right\rceil$$

La profondeur cible (distance minimale en actions) est définie par la distance de Manhattan :

$$D^* = \left\lceil \frac{|l_0 - l_{\text{end}}| + |c_0 - c_{\text{end}}|}{3} \right\rceil$$

8.1.a Calcul de la Surface Cumulative $S_{\text{geo}}(d)$

La fonction $S_{\text{geo}}(d)$ représente le nombre total de positions géométriques théoriques dans un losange de rayon d , tronqué par les murs.

1. Contribution du Squelette (Axes) :

$$S_{\text{axes}}(d) = 3[\min(d, \delta_N) + \min(d, \delta_S) + \min(d, \delta_E) + \min(d, \delta_O)]$$

2. Contribution des Quadrants : Pour chaque quadrant Q (ex: Nord-Ouest), nous sommions les barres verticales limitées par la distance restante et les murs adjacents.

$$S_{Q(d)} = \sum_{k=1}^{\min(d, \delta_{y_Q})} 3 \times \min(\delta_{x_Q}, d - k)$$

3. Surface Totale :

$$S_{\text{geo}}(d) = 1 + S_{\text{axes}}(d) + \sum_{Q \in \{\text{NE}, \text{NO}, \text{SE}, \text{SO}\}} S_{Q(d)}$$

8.1.b Optimisation Algébrique :

La formule

$$S_{Q(d)} = \sum_{k=1}^R 3 \times \min(\delta_{x_Q}, d - k)$$

ne peut pas être simplifiée directement car l'opérateur $\min$ n'est pas linéaire. Nous devons scinder la somme en deux régimes distincts selon que la largeur est contrainte par le mur (δ_{x_Q}) ou par la distance restante ($d - k$).

Soit $R = \min(d, \delta_{y_Q})$ la hauteur totale explorée. Nous identifions le point de bascule k_{lim} où la largeur naturelle $d - k$ passe sous la barre du mur δ_{x_Q} :

$$d - k < \delta_{x_Q} \iff k > d - \delta_{x_Q}$$

Nous définissons alors deux hauteurs partielles :

1. **Hauteur Saturée (h_{sat})** : Nombre de termes où le mur limite la largeur (somme constante).

$$h_{\text{sat}} = \max(0, \min(R, d - \delta_{x_Q}))$$

2. **Hauteur Libre (h_{lib})** : Nombre de termes restants (somme arithmétique).

$$h_{\text{lib}} = R - h_{\text{sat}}$$

La somme devient :

$$S_{Q(d)} = \underbrace{3 \times h_{\text{sat}} \times \delta_{x_Q}}_{\text{Partie Rectangle}} + \underbrace{3 \times h_{\text{lib}} \times \frac{(d - h_{\text{sat}} - 1) + (d - R)}{2}}_{\text{Partie Trapèze}}$$

8.2 Synthèse : Espérance du Coût Global

Nous pouvons désormais assembler les trois composantes du modèle :

1. La **probabilité d'arrêt** : Le BFS s'arrête en moyenne après avoir exploré 50% de la dernière couche D^* .
2. La **géométrie bornée** : Calculée via S_{geo} .
3. La **réduction par obstacles** : Modélisée par le facteur de Bernoulli $(1 - 4\rho)$.

L'espérance du nombre de sommets effectivement traités par le BFS pour atteindre une cible à distance D^* est :

$$E[V_{\text{BFS}}] = \underbrace{4}_{\text{nombre d'états d'un noeud}} \times \underbrace{(1 - 4\rho)}_{\text{Densité obstacles}} \times \underbrace{\frac{S_{\text{geo}}(D^*) + S_{\text{geo}}(D^* - 1)}{2}}_{\text{Moyenne Probabiliste (Arrêt à mi-couche)}}$$

$$E[V_{\text{BFS}}] = 2 \times (1 - 4\rho) \times (S_{\text{geo}}(D^*) + S_{\text{geo}}(D^* - 1))$$

8.3 Conclusion et Modèle de Coût du BFS

Afin d'établir un critère de décision rigoureux, nous formalisons ici le coût espéré exact de l'algorithme BFS. Contrairement à une approche simpliste considérant le coût par sommet constant, nous intégrons la variation de la densité topologique démontrée précédemment.

8.3.a 1. Coût Unitaire Dynamique ($C_{\text{BFS}}(P)$)

Le coût de traitement d'un sommet par le BFS est proportionnel au nombre de voisins explorés (degré).

Le coût unitaire correspond au **degré moyen** du graphe résiduel, noté $\delta(P)$:

$$C_{\text{BFS}}(P) \approx \delta(P) = \frac{20V_0 - 104P}{4V_0 - 16P}$$

Analyse de la décroissance : Pour comprendre l'évolution de ce coût, observons le « coût marginal » d'un obstacle :

- Un obstacle supprime 16 sommets.
- Il supprime environ 104 arêtes (sortants + entrants).
- Le ratio local de suppression est de $\frac{104}{16} = 6.5$.

8.3.b 2. Espérance du Coût Total ($E[\text{Cost}]_{\text{BFS}}$)

Le coût total est le produit de ce coût unitaire dynamique par le volume de sommets explorés. Le modèle intègre :

1. **Le Coût Structurel** : La densité décroissante $\delta(P)$.
2. **La Validité Topologique** : La probabilité $(1 - 4\rho)$ qu'un sommet soit libre.
3. **Le Volume Géométrique** : La surface moyenne explorée bornée par les murs (S_{geo}).

$$E[\text{Cost}]_{\text{BFS}} \approx \underbrace{\delta(P)}_{\text{Coût unitaire décroissant}} \times \underbrace{2}_{\text{Résidu des états}} \times \underbrace{(1 - 4\rho)}_{\text{Densité validité}} \times \left(\underbrace{S_{\text{geo}}(D^*) + S_{\text{geo}}(D^* - 1)}_{\text{Volume Géométrique}} \right)$$

8.3.c 3. Formulation du Ratio Critique

L'objectif final est de déterminer la rentabilité de l'algorithme A^* . Nous posons le ratio de performance R :

$$R = \frac{E[\text{Cost}]_{\text{BFS}}}{E[\text{Cost}]_{A^*}}$$

En injectant le modèle complet, la condition de bascule ($R > 1$) devient :

$$\frac{\delta(P) \times 2 \times (1 - 4\rho) \times (S_{\text{geo}}(D^*) + S_{\text{geo}}(D^* - 1))}{E[\text{Cost}]_{A^*}} > 1$$

Cette formulation isole parfaitement les variables influençant la décision :

- $\delta(P)$ capture l'impact structurel (baisse de connectivité).
- $(1 - 4\rho)$ capture l'impact volumique (réduction de l'espace).
- S_{geo} capture les contraintes géométriques de l'entrepôt (murs).

La section suivante s'attachera à modéliser le terme $E[\text{Cost}]_{A^*}$ pour résoudre cette inéquation.

9 Analyse Fine A^* : (Cycle RC)

Pour capturer la réalité physique de l'entrepôt, nous modélisons l'évolution du taux de pièges $\psi(\rho)$ comme un **cycle complet de Charge et Décharge**. La complexité topologique n'est pas monotone : elle augmente jusqu'à un point de rupture (percolation), puis s'effondre par fusion massive des obstacles.

9.1 1. Le Phénomène Physique

Nous identifions deux forces concurrentes qui agissent sur la topologie du graphe :

1. **Force Constructive (Charge)** : L'ajout d'obstacles crée de nouvelles connexions et des formes concaves (« U », « V »).
2. **Force Destructive (Décharge)** : Au-delà d'une certaine densité, l'ajout d'obstacles « bouche » les concavités et fusionne les murs séparés en blocs massifs.

Le point de bascule est le **Seuil de Percolation** :

$$\rho_c \approx \frac{0.40}{4}$$

On divise par 4 car un obstacle enlève 4 sommets physique, les sommets obtenu par des rotation n'impacte pas la connexité donc nous ne les prenons pas en compte.

9.2 2. Modélisation Mathématique (Fonction par Morceaux)

Nous définissons le taux de concavité $\psi(\rho)$ comme la tension aux bornes d'un condensateur soumis à une impulsion.

9.2.a Phase 1 : Charge Capacitive ($0 < \rho \leq 0.1$)

Modélisation (Analogie RC) : Cette phase correspond à la « charge » du système en contraintes. Nous modélisons la probabilité de formation des pièges par une cinétique de saturation. L'équation prend la forme d'une charge de condensateur, régie par une constante de densité caractéristique τ_{charge} .

$$\psi_{\text{charge}}(\rho) = 1 - \exp\left(-\left(\frac{\rho}{\tau_{\text{charge}}}\right)^2\right)$$

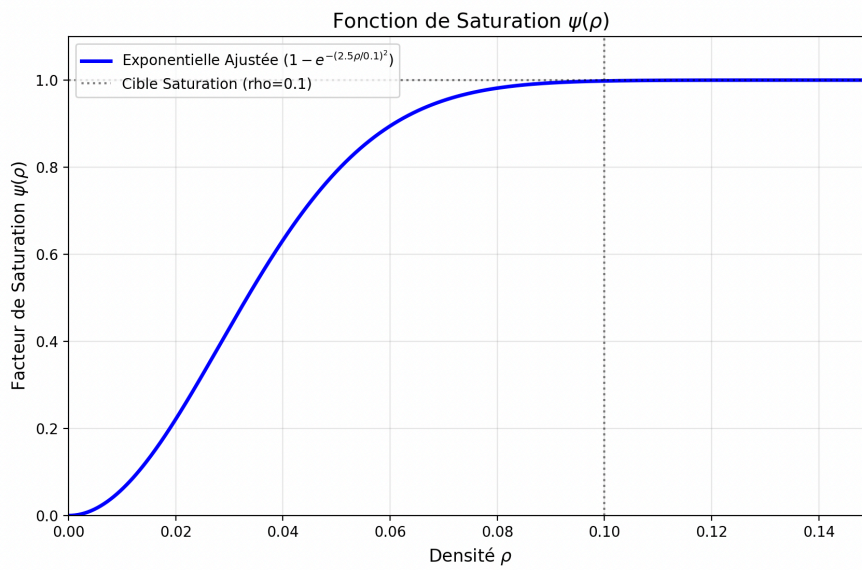


Fig. 1. – Fonction de croissance du nombre d'obstacle concave.

Détermination de τ_{charge} : Le paramètre τ_{charge} est dérivé de la borne critique $\rho_c = 0.1$. Pour assurer que le système soit saturé (c'est-à-dire $\psi \approx 1$) lorsque l'on atteint cette borne, nous appliquons un facteur de normalisation de 2.5 (correspondant à une saturation à 99.8%).

$$\tau_{\text{charge}} = \frac{\text{Borne Critique}}{\text{Facteur de Saturation}} = \frac{0.1}{2.5} = 0.04$$

Interprétation : $\tau_{\text{charge}} = 0.04$ représente l'inertie initiale du système. Le terme quadratique traduit l'effet d'accélération par interaction de paires.

9.2.b Phase 2 : Décharge Résistive ($\rho > 0.1$)

Au-delà de la densité critique, le système « se décharge ». L'espace navigable s'effondre exponentiellement. Cette dynamique est pilotée par une constante de temps de chute τ_{chute} .

$$\psi_{\text{décharge}}(\rho) = \exp\left(-\frac{\rho - 0.1}{\tau_{\text{chute}}}\right)$$

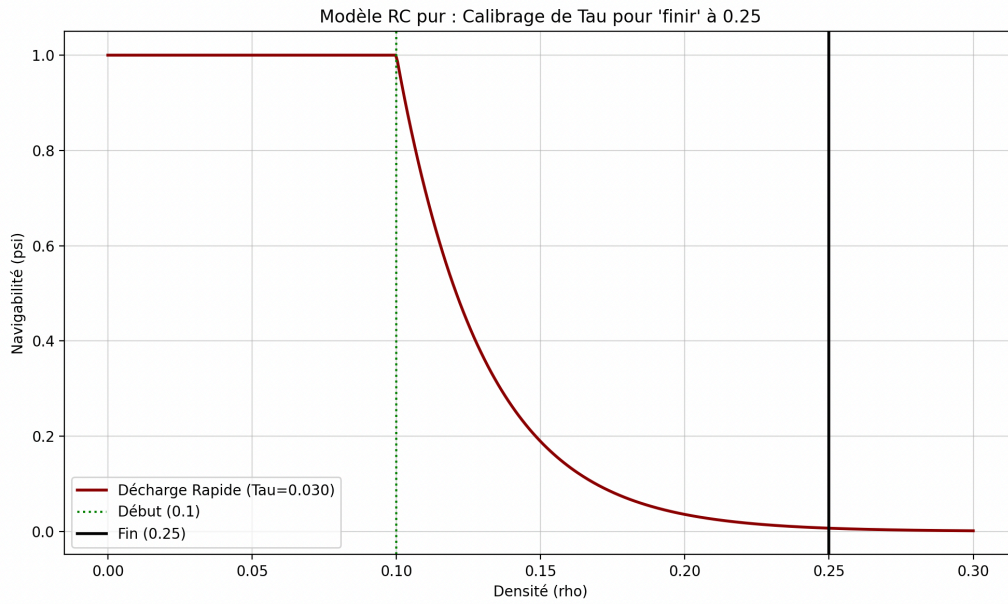


Fig. 2. – Fonction de décroissance du nombre d'obstacle concave.

Détermination de τ_{chute} : Ce paramètre est calculé en fixant une condition stricte d'arrêt du système à la borne $\rho = 0.25$ car on fait $4 \times \rho$ pour avoir la densité la densité réel. Nous utilisons la règle physique des « 5 constantes de temps » (5τ) nécessaire pour une dissipation complète du signal ($\psi \approx 0$).

1. **Plage de décharge** : $\Delta\rho = 0.25 - 0.1 = 0.15$.
2. **Facteur de dissipation** : 5 (critère de convergence à zéro).

$$\tau_{\text{chute}} = \frac{\text{Plage Disponible}}{\text{Facteur Dissipatif}} = \frac{0.15}{5} = 0.03$$

Interprétation : Une valeur faible de $\tau_{\text{chute}} = 0.03$ indique une décroissance brutale. Le système perd sa navigabilité extrêmement vite dès que le seuil de 0.1 est franchi.

9.3 3. Synthèse de la Fonction de Navigabilité

L'équation d'état globale du système $\psi(\rho)$, est définie par :

$$\psi(\rho) = \begin{cases} 1 - \exp\left(-\left(\frac{\rho}{0.04}\right)^2\right) & \text{si } 0 \leq \rho \leq 0.1 \\ \exp\left(-\frac{\rho-0.1}{0.03}\right) & \text{si } 0.1 \leq \rho \leq 0.4 \end{cases}$$

10 Modèle Final : Le Mélange de Régimes

Le passage du régime fluide au régime critique n'est pas additif, mais substitutif. Lorsque le réseau sature ($\rho \rightarrow \rho_c$), les segments linéaires disparaissent totalement. On ne fait pas « du linéaire PLUS du quadratique », on bascule « DU linéaire VERS le quadratique ».

Nous modélisons l'espérance mathématique en utilisant $\psi(\rho)$ comme un **coefficient de mélange** entre les deux modes de transport.

$$E[N_{A^*}] \approx \underbrace{(1 - \psi(\rho))[D^*(1 + \alpha\rho)]}_{\text{Mode Convexe (Disparaît à saturation)}} + \underbrace{\psi(\rho)[D^* \times S_{\text{BFS}}]}_{\text{Mode Concave (Domine à saturation)}}$$

où α désigne le coefficient d'impact associé aux obstacles convexes.

10.1 Analyse de la Transition de Phase

Cette formulation capture parfaitement la physique du changement d'état :

1. **Régime Fluide** ($\rho \approx 0$) : $\psi \approx 0$. Le terme de droite s'annule. Le terme de gauche est à 100%.

$$E[N] \approx D^*$$

→

L'algorithme est purement linéaire.

2. **Seuil de Percolation** ($\rho \approx \rho_c$) : La probabilité de piège est maximale ($\psi \approx 1$).

- ▶ Le terme linéaire ($1 - \psi$) s'éteint : il n'y a plus de chemins simples.
- ▶ Le terme quadratique est à 100% : le robot est constamment en train d'inonder des culs-de-sac.

$$E[N] \approx D^* \times S_{\text{BFS}}$$

→

L'algorithme est purement quadratique (pire cas).

3. **Régime Bloqué** ($\rho > \rho_c$) : Le graphe est brisé. Bien que ψ diminue (fusion), l'espace accessible S_{BFS} (qui contient $1 - \rho$) tend vers 0. L'espérance totale s'effondre logiquement vers 0.

10.2 5. Interprétation des 3 Régimes

1. **Régime Fluide** ($\rho < 5\%$) : Nous sommes au début de la charge (ρ^2 très faible). $\psi \approx 0$. A^* est en régime linéaire pur. C'est la zone de confort.
2. **Régime Critique** ($5\% < \rho < 10\%$) : Nous sommes au sommet de la courbe (Autour de ρ_c). ψ est maximal. Les pièges sont nombreux et complexes. C'est le **Pire Cas**. La complexité explose vers le quadratique.
3. **Régime Bloqué** ($\rho > 10\%$) : Nous sommes dans la décharge rapide. ψ chute. Paradoxalement, le nombre de pièges **navigables** diminue car le graphe devient un bloc solide. L'algorithme échoue très vite (car S_{BFS} tend aussi vers 0 via le terme $1 - \rho$).

11 Modélisation de la Distance Effective : La Divergence Critique

Jusqu'à présent, nos modèles reposaient sur la distance théorique de Manhattan D^* , qui suppose un chemin optimal quasi-rectiligne. Or, la réalité physique de l'entrepôt est bien différente : l'ajout d'obstacles n'impose pas de simples pénalités additives, mais modifie profondément la topologie de l'espace.

Nous introduisons ici la notion de **Distance Effective** $D_{\text{eff}(\rho)}$, correspondant à la distance réelle parcourue par le robot pour contourner les structures d'obstacles.

11.1 1. Le Facteur de Tortuosité $\tau(\rho)$

La distance effective est définie comme la distance à vide dilatée par un coefficient de complexité structurelle, appelé **tortuosité** :

$$D_{\text{eff}(\rho)} = D^* \times \tau(\rho)$$

Le comportement de $\tau(\rho)$ est dicté par la proximité avec la densité critique de blocage $\rho_{\text{max}} = 0.25$ (densité où l'espace devient totalement hermétique).

- **En régime dilué ($\rho \approx 0$)** : Les obstacles sont isolés. Le robot effectue de légers contournements locaux. L'augmentation du chemin est marginale et quasi-linéaire.
- **En régime dense ($\rho \rightarrow \rho_{\max}$)** : Les obstacles fusionnent pour former des barrières macroscopiques. Le robot est contraint à de longs détours (backtracking) pour trouver les rares goulots d'étranglement restants. La distance n'augmente plus linéairement, elle explose.

11.2 2. Loi de Divergence Asymptotique

Pour modéliser cette transition brutale d'une croissance lente vers une croissance verticale, nous utilisons une loi de puissance avec singularité, analogue aux modèles de percolation critiques en physique des matériaux :

$$\tau(\rho) = 1 + \beta \left[\left(1 - \frac{\rho}{\rho_{\max}} \right)^{-\mu} - 1 \right]$$

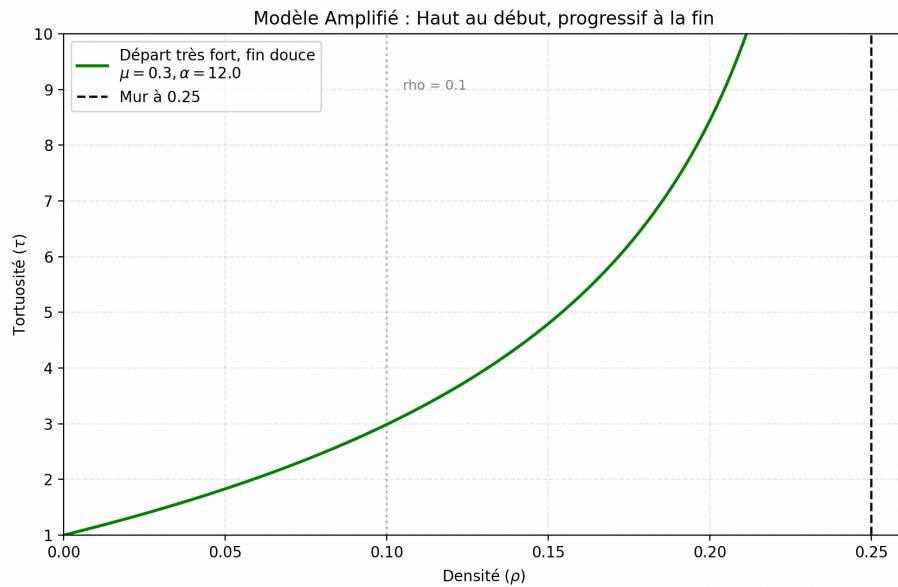


Fig. 3. – Divergence de la distance effective à l'approche de la densité critique.

Où :

- $\rho_{\max} = 0.25$ est la densité limite (asymptote verticale) où le chemin devient infini (le graphe n'est plus connexe).
- μ est l'**exposant critique** (fixé empiriquement à $\mu \approx 1.5$), contrôlant la raideur de la courbure « exponentielle ».
- β est le **facteur d'échelle**, contrôlant l'amplitude de la pénalité lorsque la densité augmente.

11.3 3. Conséquence sur le Modèle de Coût

Cette correction est fondamentale pour la précision du critère de choix

Dans la formule finale du ratio R , nous remplacerons donc désormais le terme statique D^* par sa version dynamique $D_{\text{eff}}(\rho)$.

12 Justification Physique du Modèle : Analogie avec la Percolation

Notre modélisation par cycle de Charge/Décharge correspond au comportement de la **Probabilité d'Appartenance au Cluster Géant** décrit par la Théorie de la Percolation.

Note Importante : La fonction $\psi(\rho)$ définit ici une **probabilité instantanée de piège** pour une densité donnée, et non une densité de probabilité sur l'espace des densités. L'aire sous la courbe ne doit donc pas être normalisée à 1 ; c'est l'amplitude $\psi_{\max}$ qui est bornée par 1.

12.1 1. Le Phénomène de Transition de Phase

Le réseau d'obstacles subit une transition de phase au seuil critique $\rho_c \approx 0.10$.

- **Phase « Sol » (Charge) :** Les obstacles s'agglomèrent lentement. La complexité monte progressivement.
- **Phase « Gel » (Décharge) :** Au-delà du seuil, l'espace se ferme brutalement. La chute de la navigabilité est beaucoup plus raide que la montée de la complexité.

12.2 Modélisation du Coût Structurel de A^*

Contrairement au BFS dont le coût de traitement par sommet est constant (simple ajout en file FIFO), l'algorithme A^* paie une taxe d'organisation dynamique à chaque étape. Ce coût provient de la gestion du Tas Binaire qui doit rester trié.

La complexité ne dépend pas explicitement de la profondeur P ou du degré moyen δ dans la formule finale, car ces facteurs influencent directement la valeur de N_{A^*} (le nombre de nœuds découverts). Le coût réel est celui du tri de ces N_{A^*} nœuds.

12.2.a 1. La Somme des Logarithmes

Soit N_{A^*} le volume total de sommets explorés par l'algorithme pour converger. Insérer le k -ième élément dans la file de priorité coûte $\log_2(k)$.

Le coût total de l'algorithme est donc la somme cumulée des coûts d'insertion pour tous les nœuds découverts :

$$E[\text{Cost}]_{A^*} = \sum_{k=1}^{N_{A^*}} \log_2(k)$$

Par propriété des logarithmes ($\log a + \log b = \log(a \times b)$), cette somme se simplifie en une factorielle :

$$E[\text{Cost}]_{A^*} \approx \log_2(N_{A^*}(\rho)!)$$

Note : Via l'approximation de Stirling, cela revient à une complexité de l'ordre de $O(N \log N)$, mais la forme factorielle $\log(N!)$ est plus précise pour les petites valeurs.

12.3 Analyse du Ratio de Performance R (Modèle Avancé)

L'objectif est d'exprimer R en injectant les modèles de coût développés précédemment.

12.3.a Développement du Dénominateur (Le Coût A^*)

Le coût de A^* est logarithmique par rapport au nombre de nœuds visités N_{A^*} .

$$E[\text{Cost}]_{A^*} \approx \log_2([N_{A^*}]!)$$

Nous injectons le modèle de mélange de régimes. Le volume quadratique du « Régime Concave » correspond exactement au volume exploré par le BFS (calculé ci-dessus).

$$N_{A^*} = \underbrace{(1 - \psi(\rho)) [D_{\text{eff}(\rho)}(1 + \alpha\rho)]}_{\text{Régime Linéaire}} + \underbrace{\psi(\rho) [E[V_{\text{BFS}}]]}_{\text{Régime Concave}}$$

D'où :

$$E[\text{Cost}]_{A^*} = \log_2 \left(\left[(1 - \psi(\rho)) D_{\text{eff}(\rho)} (1 + \alpha\rho) + \psi(\rho) \times 2(1 - 4\rho) S_{\text{geo}}(D_{\text{eff}(\rho)}) + S_{\text{geo}}(D_{\text{eff}(\rho)} - 1) \right]! \right)$$

12.3.b Étape 3 : La Formule du Ratio Critique

En combinant les deux modélisations, nous obtenons l'équation maîtresse du ratio R . Pour la lisibilité, posons $S_{\text{moy}} = 2 \times (S_{\text{geo}}(D_{\text{eff}(\rho)}) + S_{\text{geo}}(D_{\text{eff}(\rho)} - 1))$.

$$R = \frac{\left(\frac{5(4V_0) - 104P}{4V_0 - 16P} \right) \times (1 - 4\rho) \times S_{\text{moy}}}{\log_2 \left(\left[(1 - \psi(\rho)) D_{\text{eff}(\rho)} (1 + \alpha\rho) + \psi(\rho) (1 - 4\rho) S_{\text{moy}} \right]! \right)}$$

Analyse : Cette forme explicite montre que le numérateur subit deux forces opposées :

- Le terme $\delta(P)$ augmente le ratio (le graphe devient plus complexe/dense localement pour les survivants) donc il favorise A^* .
- Le terme $(1 - 4\rho)$ diminue le coût (il y a moins de nœuds valides globalement) ce qui favorise le BFS.

12.3.c Application Numérique (Approximation de Stirling)

Pour déterminer quel algorithme lancer en temps réel, nous appliquons l'approximation de Stirling ($\log_2(n!) \approx n \log_2 n$). Cela transforme la factorielle coûteuse en une simple multiplication, rendant le calcul du critère de décision instantané.

Le ratio opérationnel R s'écrit alors en toutes lettres :

$$R \approx \frac{\left(\frac{5(4V_0) - 104P}{4V_0 - 16P} \right) \times (1 - 4\rho) \times S_{\text{moy}}}{\left[(1 - \psi(\rho)) D^* (1 + \alpha\rho) + \psi(\rho) (1 - 4\rho) S_{\text{moy}} \right] \times \log_2 \left(\left[(1 - \psi(\rho)) D^* (1 + \alpha\rho) + \psi(\rho) (1 - 4\rho) S_{\text{moy}} \right] \right)}$$

Règle de Décision Finale : Il suffit d'évaluer cette expression au début de la mission :

- **Si $R > 1$ (Zone Fluide) :** Le coût du BFS (numérateur) est supérieur à celui de A^* . L'heuristique joue son rôle et permet d'éviter suffisamment de nœuds pour compenser le coût du tri. → **Choix Optimal : A^***
- **Si $R \leq 1$ (Zone Critique/Saturée) :** Le coût de A^* (dénominateur) dépasse celui du BFS. À cause de la densité d'obstacles ($\psi \approx 1$), A^* est forcé d'explorer un volume similaire au BFS, mais avec la pénalité logarithmique du tri. → **Choix Optimal : BFS**

13 Analyse des Performances (Benchmark)

Nous avons mené une campagne d'essais numériques pour valider les modèles théoriques asymptotique. Les résultats présentés correspondent à la médiane des temps d'exécution sur **10 instances aléatoires** par point de mesure.

13.1 1. Impact de la taille de la grille ($N \times M$)

Cette première analyse évalue la scalabilité. La taille varie de 10×10 à 50×50 , avec une complexité relative maintenue constante (obstacles proportionnels).

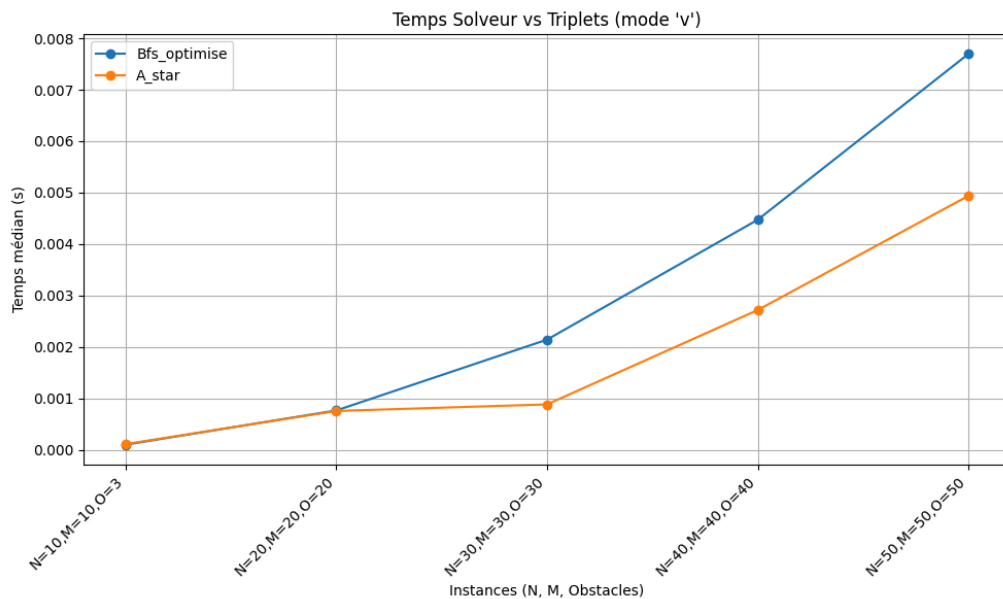


Fig. 4. – Temps moyen d'exécution en fonction de la taille ($N \times M$).

Analyse : La divergence de performance est nette dès $N = 30$.

- Sur les petites instances, le coût fixe de la file de priorité pénalise A^* , qui fait jeu égal avec le BFS.
- Sur les grandes instances (50×50), A^* devient **37% plus rapide**. L'heuristique joue pleinement son rôle d'élagage face à l'explosion quadratique de l'espace de recherche du BFS.

13.2 2. Impact de la densité locale (Grille 20×20)

Ici, la grille est fixée à 20×20 . Nous faisons varier le nombre d'obstacles de 10 à 50.

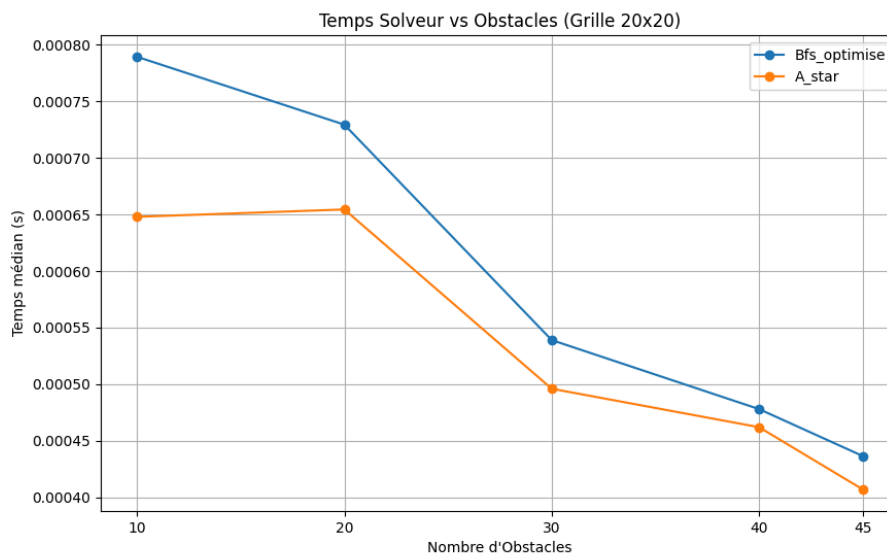


Fig. 5. – Impact de la densité d'obstacles sur une petite grille (20×20).

Analyse : On observe une **baisse globale du temps de calcul** quand le nombre d'obstacles augmente. Cela valide notre modèle. À 50 obstacles, la réduction de l'espace de recherche est telle qu'elle compense largement la complexification des chemins.

13.3 3. Analyse Avancée : Seuil de Percolation (Grille 100×1000)

Pour observer les limites macroscopiques du système, nous avons testé une grille massive (100×1000) en poussant le nombre d'obstacles jusqu'à 12 000.

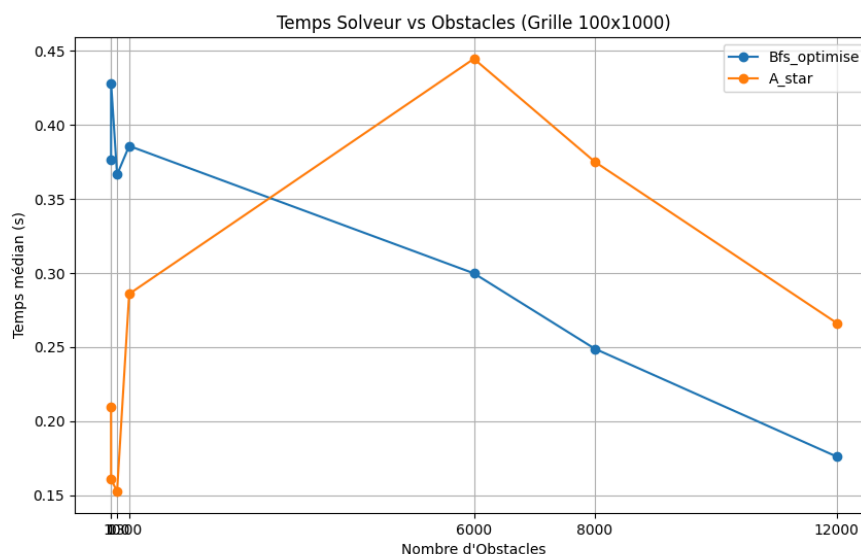


Fig. 6. – Mise en évidence du seuil de percolation et de l'effondrement du temps de calcul.

Analyse du phénomène d'effondrement : La courbe Fig. 6 présente un profil caractéristique de **transition de phase** :

1. **Phase de Complexification (0 à 6000 obstacles) :** Le temps de calcul augmente fortement (pic à $\approx 0.45s$). Le graphe est connexe mais devient labyrinthique (« tortueux »), mettant en défaut l'heuristique de A^* .
2. **Phase d'Effondrement (> 8000 obstacles) :** On observe une chute brutale des temps de calcul. À 12 000 obstacles, le temps redevient très faible ($< 0.20s$).

Interprétation Physique : Piège de l'Heuristique et Backtracking

Puisque l'existence d'un chemin est garantie par le protocole de test, l'augmentation du temps de calcul (pic observé à 6000 obstacles) ne provient pas d'une rupture de connexité, mais d'une **défaillance de l'heuristique $h(n)$** .

1. **Leurre Heuristique :** À haute densité, les obstacles forment des structures en « U » ou des culs-de-sac. La distance de Manhattan (ou Euclidienne) continue d'indiquer que la cible est « tout droit », incitant A^* à s'engouffrer dans ces zones bloquées.
2. **Coût du Backtracking :** Pour sortir de ces pièges locaux (minima locaux), l'algorithme doit explorer exhaustivement l'intérieur du cul-de-sac avant de revenir en arrière. On effectue alors un nombre massif d'opérations d'insertion/extraction.
3. **Sanction Logarithmique :** C'est ici que la complexité théorique nous rattrape. Contrairement au BFS qui gère ses retours en arrière en temps constant $O(1)$, A^* paie le prix fort : chaque nœud exploré inutilement dans ces phases de « backtrack » coûte $O(\log|V|)$ à cause du tri dans la file de priorité.

Note sur la chute finale : La baisse du temps observée à 12 000 obstacles s'explique par la réduction massive de la taille du graphe ($|V|$). Le nombre de nœuds supprimés (env. 12000×16) est tel que, même avec une heuristique inefficace, l'espace de recherche total devient suffisamment petit pour être

parcouru rapidement. De plus, il y a beaucoup moins de culs-de-sac, comme expliqué dans la partie bonus.

14 Modélisation du programme linéaire

On présente ici la modélisation d'un programme linéaire, visant à générer une grille $M \times N$ contenant exactement P obstacles. Les obstacles sont représentés par des variables binaires et sont soumis à plusieurs contraintes structurelles destinées à contrôler leur répartition et à éviter certains motifs indésirables.

L'objectif est de minimiser une fonction de coût aléatoire définie sur les positions de la grille, et la résolution est effectuée à l'aide du solveur Gurobi.

14.1 Variables et paramètres

- $M, N \in \mathbb{N}^*$: dimensions de la grille (nombre de lignes et de colonnes).
- $P \in \mathbb{N}$: nombre total d'obstacles à placer.
- Variables binaires $x_{\{i,j\}} \in \{0, 1\}$ représentant la présence d'un obstacle :

$$x_{\{i,j\}} = \begin{cases} 1 & \text{si la case contient un obstacle} \\ 0 & \text{sinon} \end{cases}$$

- Coûts $c_{\{i,j\}} \in [0, 1000]$ générés aléatoirement.

14.2 Modélisation du Problème de PLNE

Variables de décision :

$$x_{i,j} = \begin{cases} 1 & \text{si un obstacle est placé en } (i, j) \\ 0 & \text{sinon} \end{cases}$$

Nous formulons le problème de placement comme un Programme Linéaire en Nombres Entiers.

Fonction Objectif :

$$\text{Minimiser } Z = \sum_{i=1}^M \sum_{j=1}^N c_{i,j} \cdot x_{i,j}$$

Sous contraintes :

$$\left\{ \begin{array}{ll} (1) & \sum_{i=1}^M \sum_{j=1}^N x_{i,j} = P \quad (\text{Total}) \\ (2) & \sum_{j=1}^N x_{i,j} \leq \frac{2P}{M} \quad (\text{Densité max par ligne}) \\ (3) & \sum_{i=1}^M x_{i,j} \leq \frac{2P}{N} \quad (\text{Densité max par colonne}) \\ (4) & x_{i,j} - x_{i,j+1} + x_{i,j+2} \leq 1 \quad \forall i, \forall j \in \{0, \dots, N-2\} \quad (\text{Anti-motif 101 horizontal}) \\ (5) & x_{i,j} - x_{i+1,j} + x_{i+2,j} \leq 1 \quad \forall i, \forall j \in \{0, \dots, N-2\} \quad (\text{Anti-motif 101 vertical}) \\ (6) & x_{i,j} \in \{0, 1\} \end{array} \right.$$

14.3 Preuve de la contrainte Anti-motif 1 – 0 – 1

Soit le triplet (a, b, c) correspondant aux variables $(x_{i,j}, x_{i,j+1}, x_{i,j+2})$. On souhaite interdire la configuration $(1, 0, 1)$.

1. Expression logique : L'interdiction du motif se traduit par la négation de la conjonction :

$$\neg(a \wedge \neg b \wedge c) \Leftrightarrow \neg a \vee b \vee \neg c$$

2. Linéarisation : Pour des variables binaires $x \in \{0, 1\}$, l'alternative « ou » impose que la somme des expressions littérales soit au moins égale à 1 :

$$(1 - a) + b + (1 - c) \geq 1$$

3. Simplification :

$$2 - a + b - c \geq 1$$

$$-a + b - c \geq -1$$

En multipliant par -1 , on inverse l'inégalité :

$$a - b + c \leq 1$$

Conclusion : On retrouve la contrainte structurelle :

$$x_{i,j} - x_{i,j+1} + x_{i,j+2} \leq 1$$

15 Bibliographie

Cette section référence les ouvrages théoriques et les ressources pédagogiques ayant servi de support à la modélisation mathématique et à l'analyse algorithmique de ce projet.

15.1 Ouvrages de Référence

- ▶ **Algorithmique (3ème édition)**, T. Cormen, C. Leiserson, R. Rivest, C. Stein. Éditions Dunod.
 - **Utilisation** : Fondements théoriques des graphes, preuves de correction pour BFS et A, analyse de complexité et structures de données (Tas Binaires pour la file de priorité).

15.2 Ressources Multimédia

- ▶ **Théorie de la Percolation et Transitions de Phase**. Plateforme YouTube.
 - **Utilisation** : Compréhension intuitive des phénomènes de seuil critique et de la formation des « clusters » géants dans les grilles aléatoires, appliquée ici à la rupture de l'heuristique.

15.3 Supports Académiques Personnels

- ▶ **Notes de Cours : Équations Différentielles et Circuits RC**. Niveau Terminale / L1.
 - **Utilisation** : Modélisation cinétique de la complexité. Analogie entre la courbe de formation des pièges topologiques $\psi(\rho)$ et la réponse temporelle d'un circuit RC (Charge capacitive et Décharge).

16 Remerciements

L'élaboration de la partie avancée de ce projet (« Partie Bonus ») a grandement bénéficié d'échanges scientifiques enrichissants.

Je tiens tout particulièrement à remercier **M. Jean-Noël Vittaut** (Maître de Conférences en IA) pour son idée d'avoir un coefficient de mixture pour ajouter de la transitivité pour le changement de phase dans le modèle du A^* .

Mes remerciements s'adressent également à **M. Corentin Boyer** (Docteur en Information Quantique) pour sa disponibilité et ses éclaircissements concernant les aspects de modélisation physique, qui ont permis d'affiner les analogies physiques présentées dans ce rapport.